

***Тема 12: Агенти и околна
свят. Видове. Функции.
Интелигентни агенти.
Категоризация на околната
среда.***



Съдържание

- Агенти и околна среда
- Рационалност
- Мярка за ефективност (Performance measure),
Околна среда (Environment), Актуатори, Сензори
- Типове околна среда
- Типове агенти

Дефиниция за AI ?!

... Същият въпрос, отново!

Какво казват Ръсел и Норвиг?

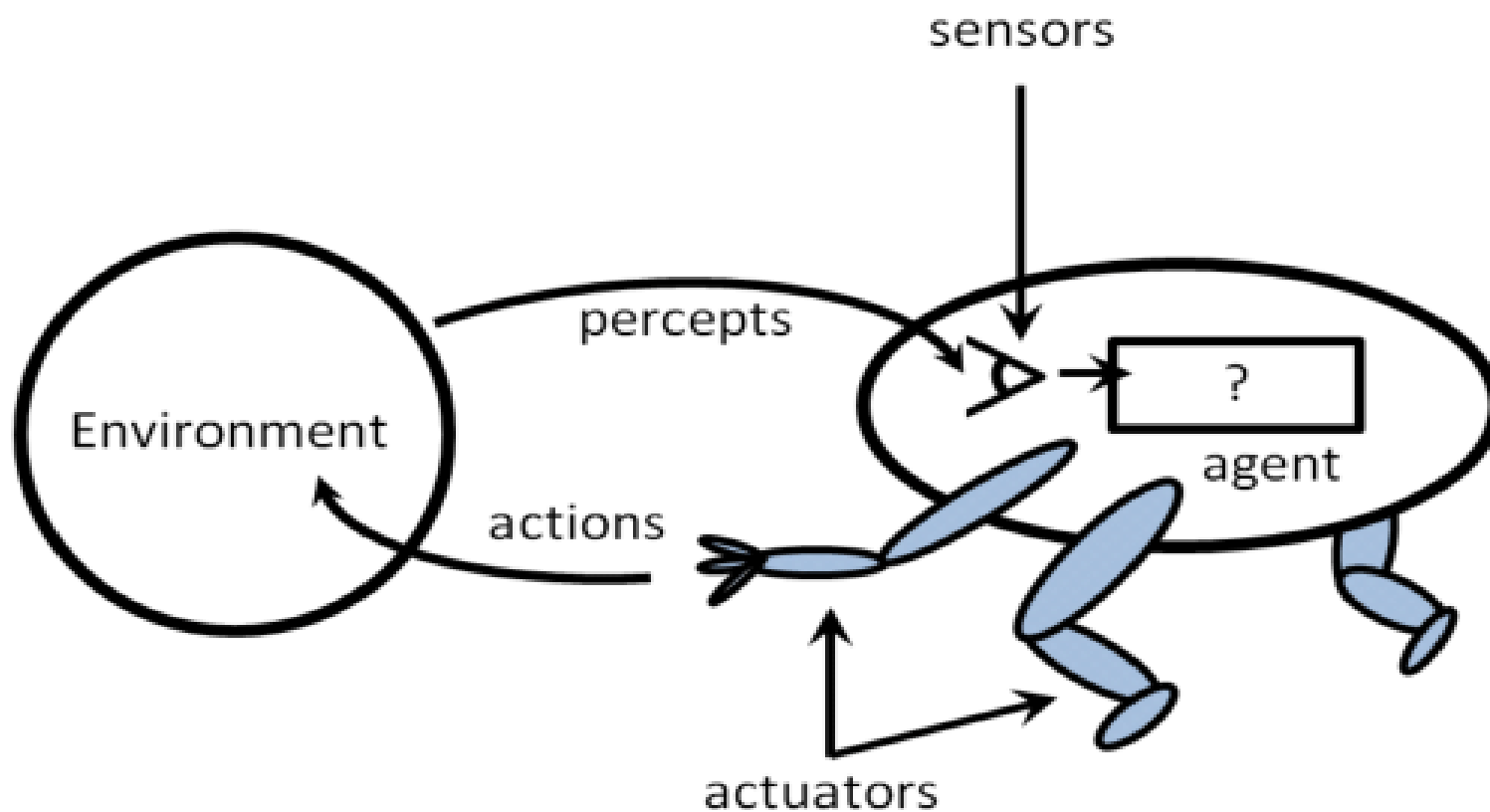
... "машини, които могат да мислят, учат и създават" ...

Гледната точка на агента на Пуул и Макуърт:

Изкуственият интелект е синтез и анализ на изчислителни агенти, които действат интелигентно.

Агенти (Ръсел и Норвиг)

- Определение за агент?
- Човек: възприятия, актуатори
- Робот: възприятия, задвижващи механизми

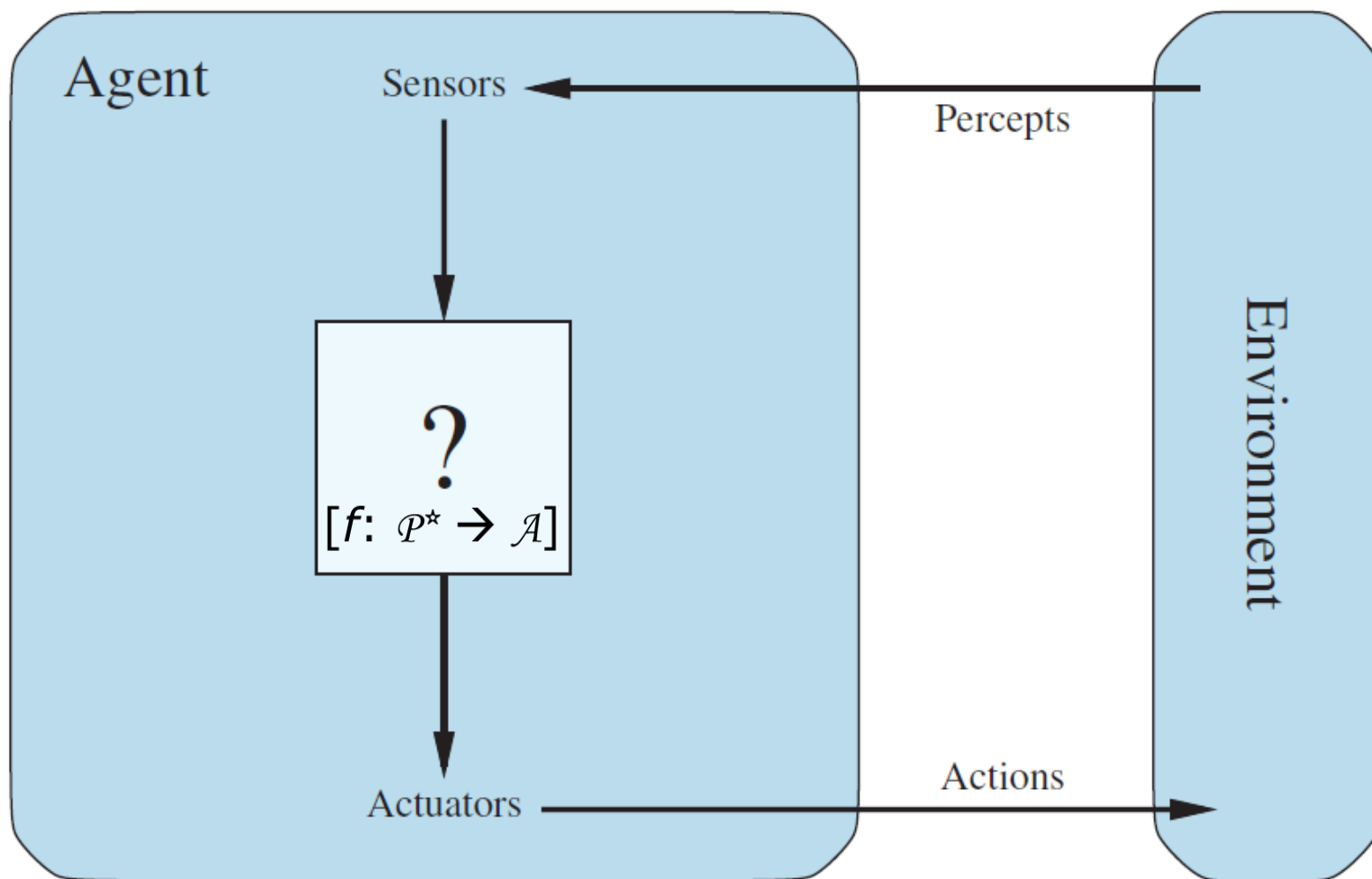


Агент

- Какво може да прави всеки агент?
 - Възприема околния свят и изпълнява действия.

- Как става това?
 - Възприема околния свят чрез сензори,
 - Интерпретира информацията въз основа на знание и информация за контекста
 - използва мярка за успешност, модели и стратегии за да взема решения,
 - решенията се прилагат чрез действия.

Агент



Агент

$$[f: P^* \rightarrow \mathcal{A}]$$

Програмата на агента се изпълнява с помощта на хардуер за да се реализира функцията f

agent = architecture + program

Кое то всъщност се свежда до

агент = хардуер + софтуер

іRobot: автоматична прахосмукачка



iRobot: автоматична прахосмукачка

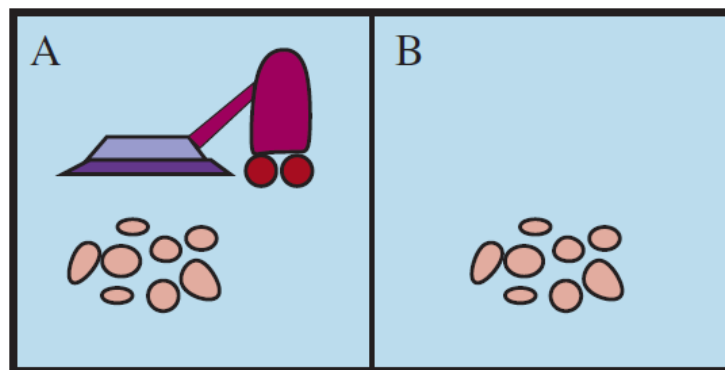


Figure 2.2 A vacuum-cleaner world with just two locations. Each location can be clean or dirty, and the agent can move left or right and can clean the square that it occupies. Different versions of the vacuum world allow for different rules about what the agent can perceive, whether its actions always succeed, and so on.

- Възприятия: местоположение и съдържание, е.г., [А, Мръсно]
- Действия: *Ляво, Дясно, Засмукване, НоОр*

iRobot: автоматична прахосмукачка



- **Рационално** поведение: това което максимизира постигането на заложената цел, при наличната към момента информация.
- Правилно е това което води към успех!!!
- Мярка за успешност: Обективен критерии за успешност на поведението на агента

За прахосмукачката:

- ☐ количеството изчистен прах,
- ☐ времето което е отнело,
- ☐ изразходваната електроенергия,
- ☐ количеството шум, и др.

Рационален агент

- **Рационален агент:**

За всяка възможна последователност от възприятия, рационалният агент ще избере действие което се очаква да максимизира избраната мярка за успешност, като се вземе в предвид наличната до момента информация и всяко друго налично знание по проблема, което агента има на разположение.

- **Рационален агент не е агент, който знае всичко, т.е. с безкрайно знание!**

Рационален агент

Рационален агент:

- Агентът може да предприема действия за да модифицира бъдещи възприятия за да получи полезна информация (сбор на информация, изследване)
- Агентът е автономен ...
- ...ако поведението му се определя от собствения му опит, способността му да се учи и адаптира.

Рационален агент:

For each possible percept sequence, a rational agent should select an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

PEAS

(Performance measure, Environment, Actuators, Sensors)

=

Мярка за ефективност, Околен свят, Изпълнителни механизми, Сензори

Пример:

Автоматичен шофьор на такси

- Performance measure? (Критерии, измерители)
- Environment? (околен свят)
- Actuators? (изпълнителни механизми)
- Sensors? (сензори)

■ Автоматичен шофьор на такси:

Agent Type	Performance Measure	Environment	Actuators	Sensors
Taxi driver	Safe, fast, legal, comfortable trip, maximize profits, minimize impact on other road users	Roads, other traffic, police, pedestrians, customers, weather	Steering, accelerator, brake, signal, horn, display, speech	Cameras, radar, speedometer, GPS, engine sensors, accelerometer, microphones, touchscreen

Figure 2.4 PEAS description of the task environment for an automated taxi driver.

Пример: Медицинска диагностична система

Agent Type	Performance Measure	Environment	Actuators	Sensors
Medical diagnosis system	Healthy patient, reduced costs	Patient, hospital, staff	Display of questions, tests, diagnoses, treatments, referrals	Keyboard entry of symptoms, findings, patient's answers

Пример: Робот за сортиране/пренасяне на части

- **Мярка за успешност:**

Процентът от частите поставени в правилните кошове (сортирани правилно)

- **Околен свят:**

Конвейерната лента с части, контейнери за части

- **Актуатори:**

Ръка с много степени на свобода (стави) и грипер (механизъм за захващане)

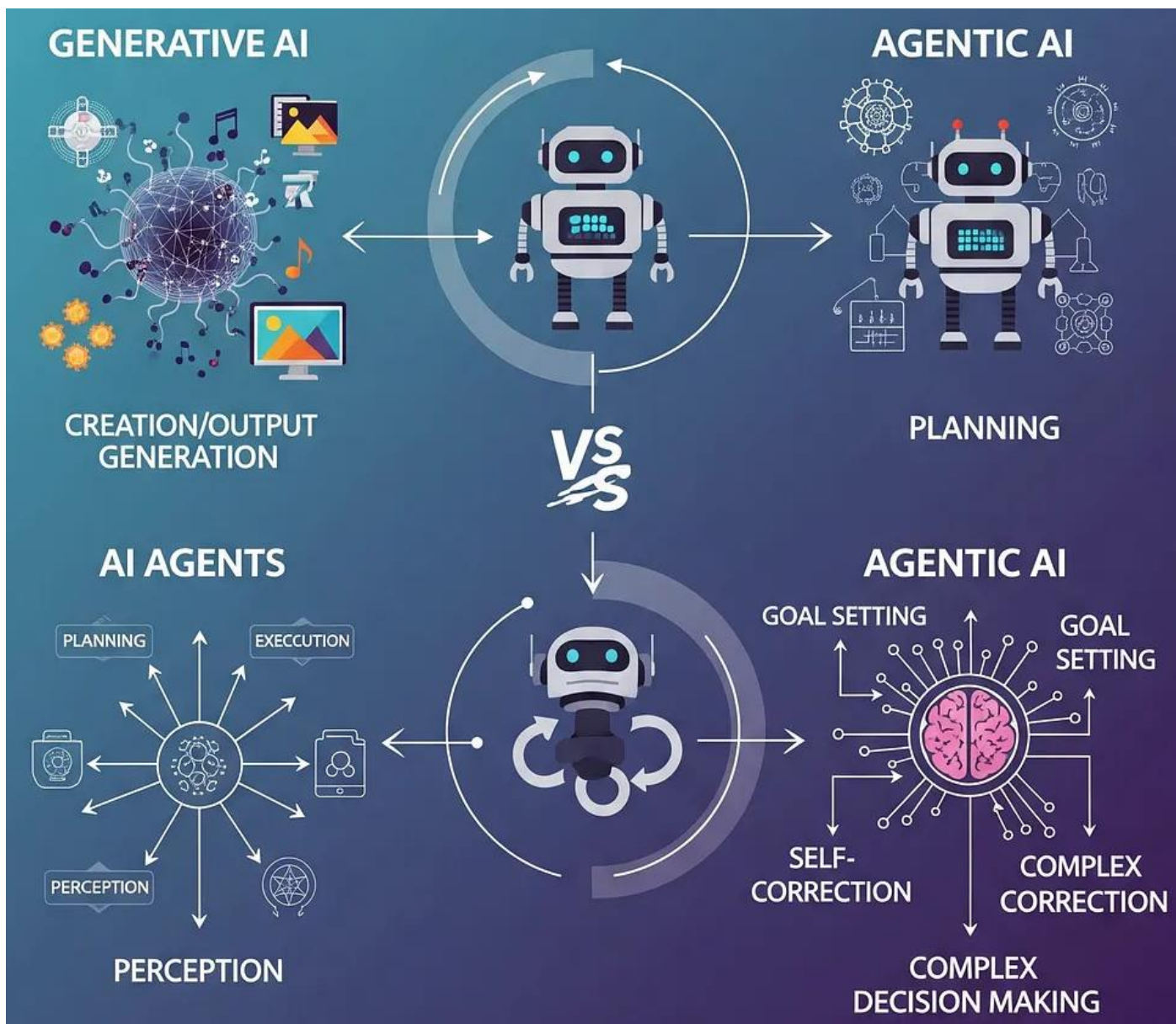
- **Сензори**

Камера, сензори за ъглово преместване и скорост на ставите и позицията на ставите и грипера

Пример: Интерактивен учител по английски език

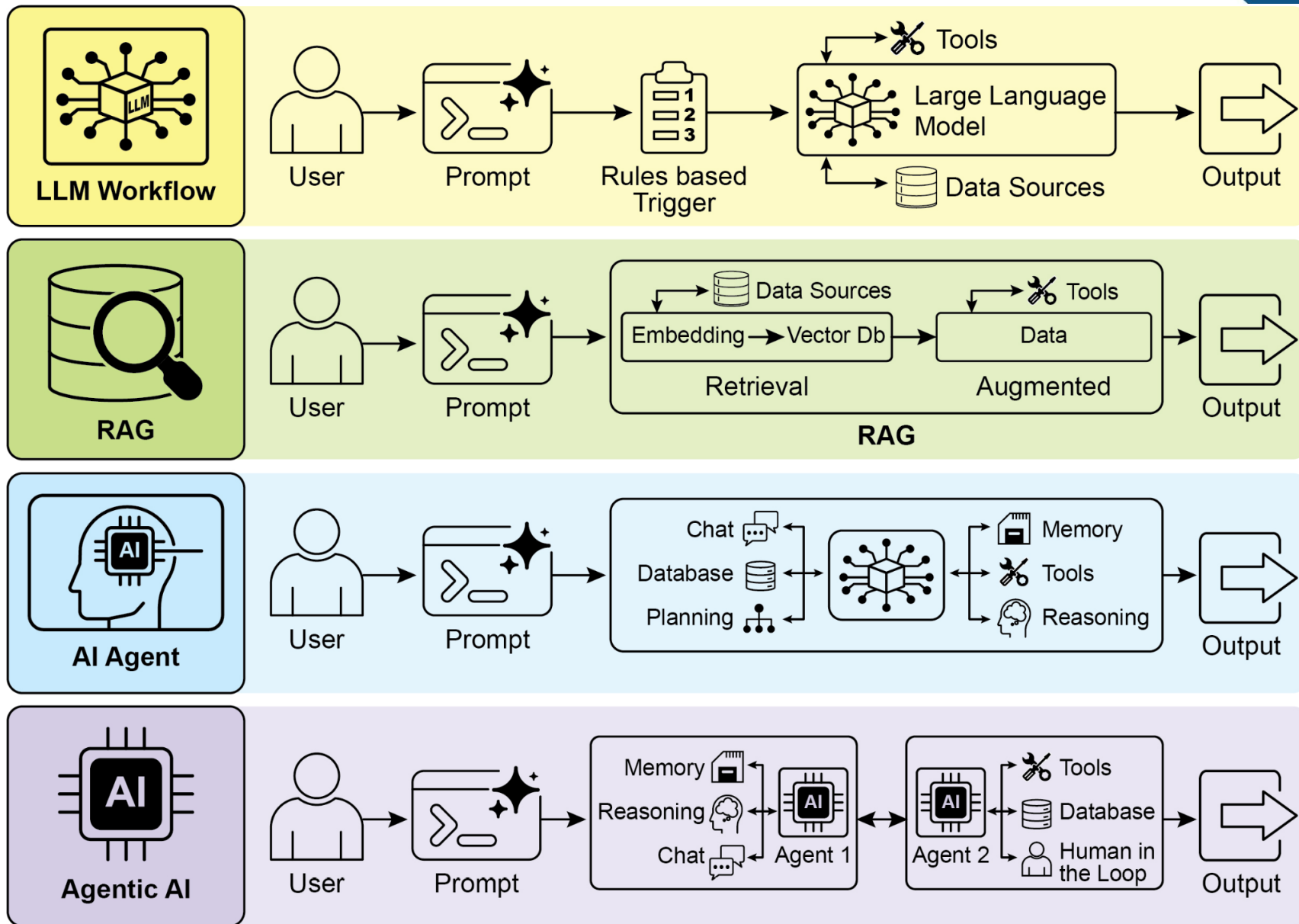
- Мярка за успешност:
Резултатите на студентите на теста
(Увеличете максимално!)
- Околен свят:
Група студенти
- Актуатори:
Дисплей (упражнения, предложения, корекции)
- Сензори:
Клавиатура, мишка

Агенти?!?!?!?





Агенти?!?!?!?



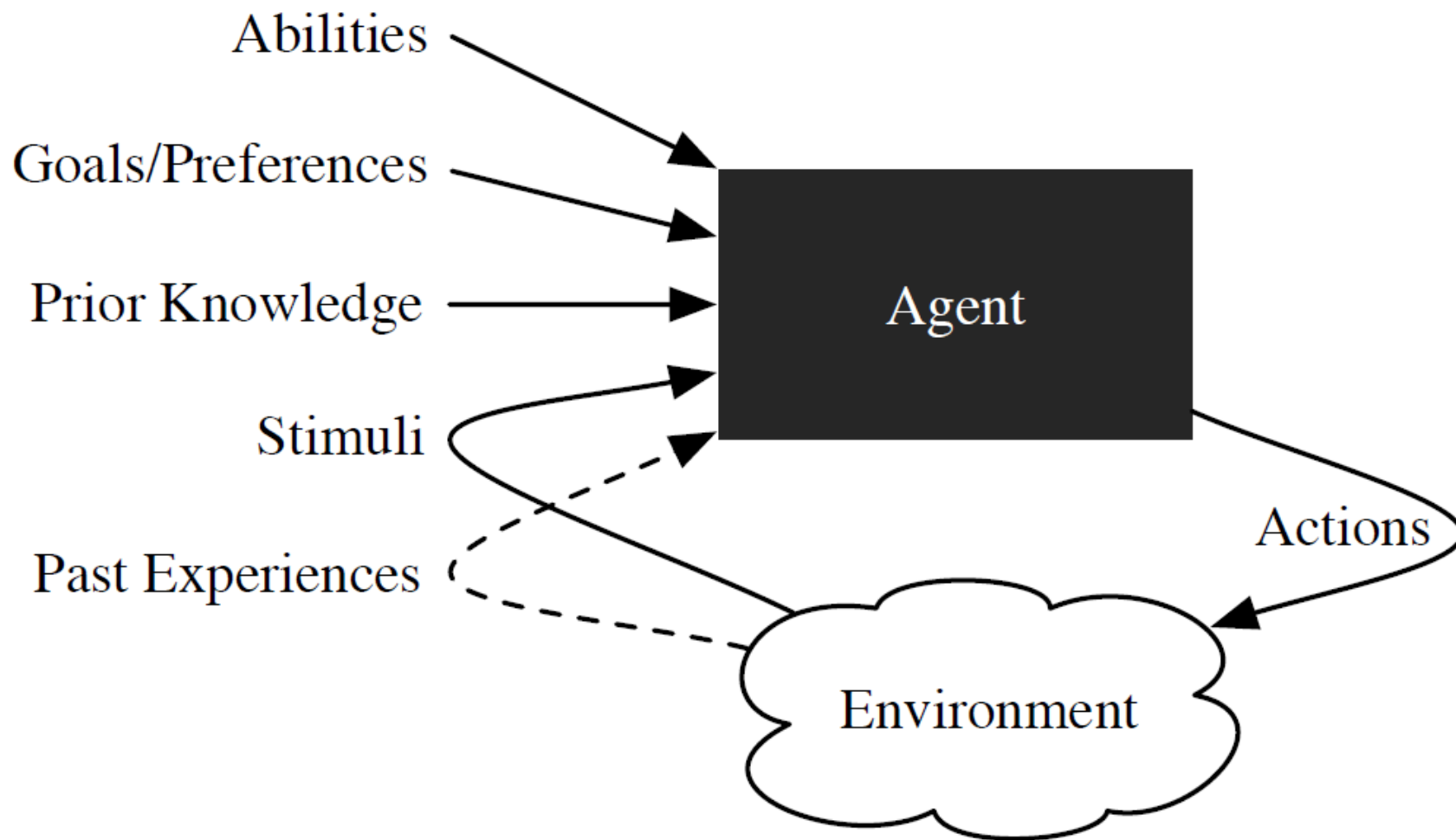
Интелигентен агент?

Агентът е нещо, което действа в оперативна среда.

Един агент действа интелигентно, ако:

- ▶ действията му са насочени към постигане на конкретна цел и са съобразени с обстоятелствата
- ▶ е гъвкав спрямо променящите се свойства на средата и/или цел,
- ▶ учи се от личния си опит, или чрез наблюдение,
- ▶ агентът прави подходящи избори, предвид присъщите му органичния във възприятията и наличните изчислителни ресурси.

Агент



Типове околна среда

- **Напълно наблюдаема** (vs. частично наблюдаема):
Сензорите на агента имат достъп то цялостното състояние на средата във всеки момент от времето.
- **Детерминистична** (vs. stochastic):
Състоянието на средата е напълно определено в рамките на всяко състояние действията на агента. Ако средата е детерминистична с изключение на действията на други агенти то средата е стратегическо
- **Епизодично** (vs. sequential):
Опитът на агента е епизодичен – във всеки епизод агента възприема и след това изпълнява едно единствено действие. Изборът на действие зависи само от конкретния епизод.

Типове околна среда

- **Статична** (vs. динамична): Средата не се променя докато агентът разсъждава какво да предприеме. (Средата е **полу-динамична** ако средата не се променя с времето но се променя поведението и производителността на агента)
- **Дискретна** (vs. непрекъснатата): Ограничен брой от разграничени, ясно дефинирани усещания и действия.
- **Един агент** (vs. мултиагент): Един агент опериращ сам в средата.

Най-труден случай: частично наблюдаема, многоагентна, стохастична, последователна, динамична, непрекъснатата и непозната среда.

Типове околна среда

Task Environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Image analysis	Fully	Single	Deterministic	Episodic	Semi	Continuous
Part-picking robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous
Refinery controller	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Interactive English tutor	Partially	Multi	Stochastic	Sequential	Dynamic	Discrete

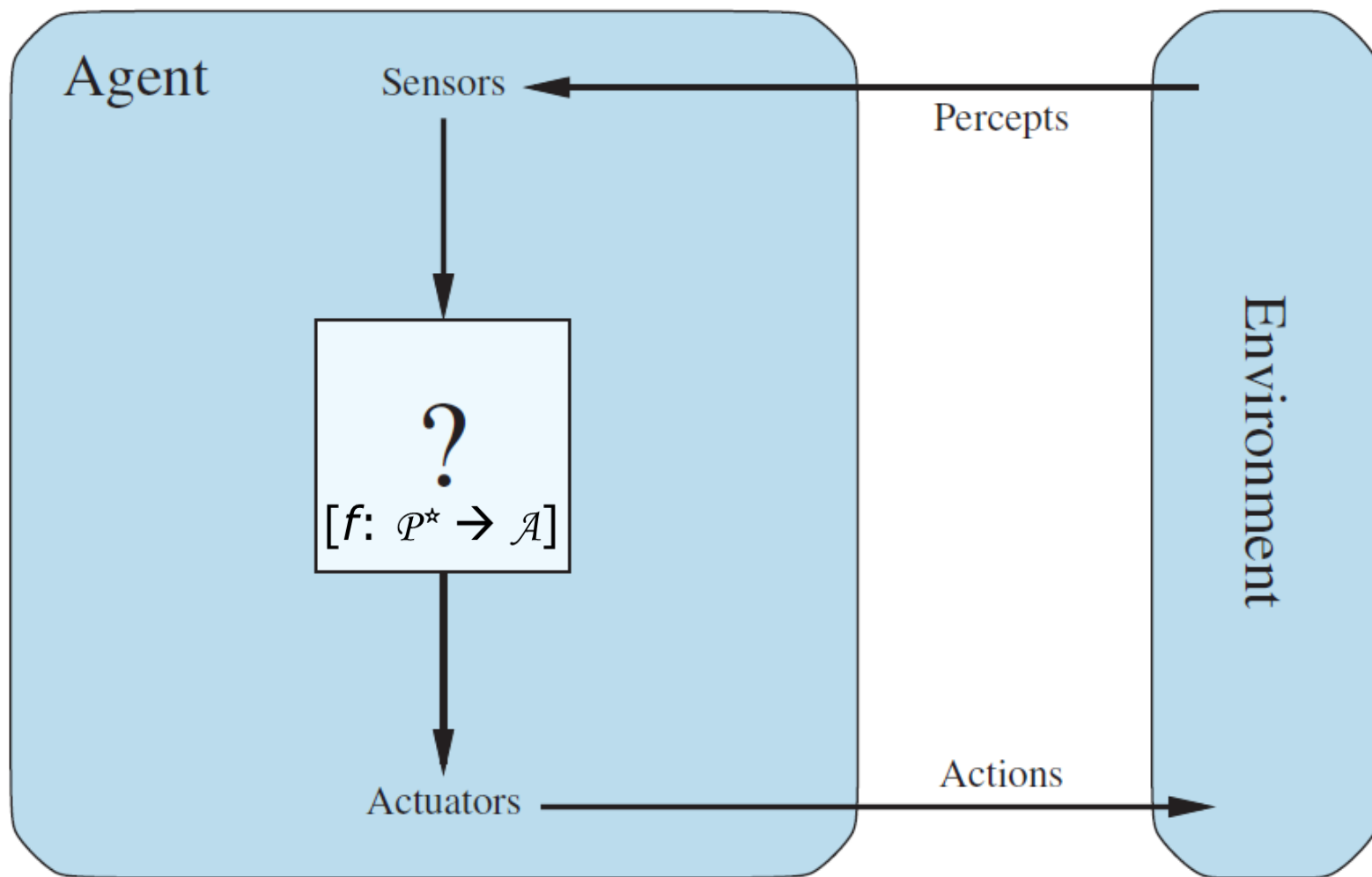
Figure 2.6 Examples of task environments and their characteristics.

Агенти - програми

Функции на агента

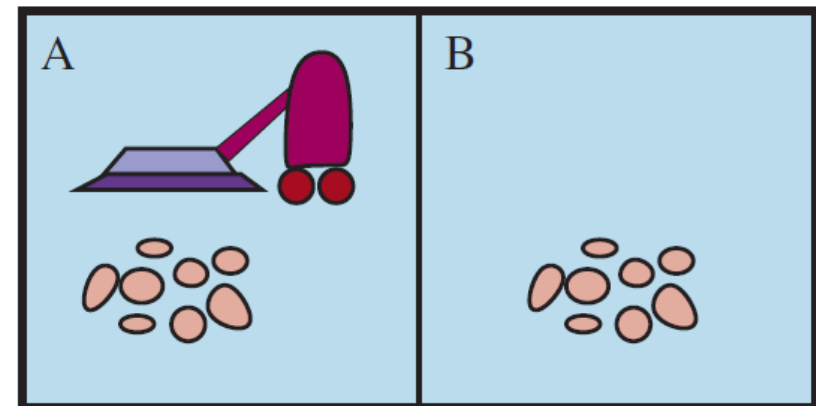
- **Агентът** е напълно определен (специфициран) от функцията си да „преобразува“ последователност от възприятия и наблюдения в действия.
- **Рационален агент:** автономна система която е има целенасочено поведение при реализирането на зададена цел.
- **Цел:** „чевръсто изпълнява доставки“

Структура на програмата Агент



Агентите взаимодействат с околната среда чрез сензори и изпълнителни механизми

Агент чрез таблица на съответствията Table-lookup agent (Таблични агенти)



Агент чрез таблица на съответствията

Table-lookup agent



Percept sequence	Action
<i>[A, Clean]</i>	<i>Right</i>
<i>[A, Dirty]</i>	<i>Suck</i>
<i>[B, Clean]</i>	<i>Left</i>
<i>[B, Dirty]</i>	<i>Suck</i>
<i>[A, Clean], [A, Clean]</i>	<i>Right</i>
<i>[A, Clean], [A, Dirty]</i>	<i>Suck</i>
<i>⋮</i>	<i>⋮</i>
<i>[A, Clean], [A, Clean], [A, Clean]</i>	<i>Right</i>
<i>[A, Clean], [A, Clean], [A, Dirty]</i>	<i>Suck</i>
<i>⋮</i>	<i>⋮</i>

Figure 2.3 Partial tabulation of a simple agent function for the vacuum-cleaner world shown in Figure ???. The agent cleans the current square if it is dirty, otherwise it moves to the other square. Note that the table is of unbounded size unless there is a restriction on the length of possible percept sequences.

Агент чрез таблица на съответствията



function TABLE-DRIVEN-AGENT(*percept*) **returns** an action

persistent: *percepts*, a sequence, initially empty

table, a table of actions, indexed by percept sequences, initially fully specified

append *percept* to the end of *percepts*

action $\leftarrow$ LOOKUP(*percepts*, *table*)

return *action*

TABLE-DRIVEN-AGENT прави това, което искаме, при условие че таблицата е попълнена правилно: тя *коректно* реализира желаната функция на агента.

Броят на записите в таблицата за търсене ще бъде:

$$\sum_{t=1}^T |\mathcal{P}|^t$$

където P е броят или възможните възприятия, а T е животът на агента (общият брой стимули, които ще получи).



Агент чрез таблица на съответствията

Нека разгледаме визуалния вход на автономно такси:

- Входът от една камера (обикновено са 8 камери) е приблизително 70 мегабайта в секунда (30 fps, 1080x720 пиксела с 24 бита цветова информация)
- ЛИДАР, инфра-червена камера, ...

Само от 8-те камери имаме таблица за търсене с над $10^{600\,000\,000\,000}$ записи за час шофиране.

Недостатъци

- Огромни таблици
- Създаването на таблицата отнема много време
- агентът не е автономен !!
- Дори когато е способен на самообучение, отнема много време да се научат отделните редове на таблицата

Така че основното предизвикателство за ИИ е да разбере как да пише софтуер, които, доколкото е възможно, генерира рационално поведение от програма, а не от огромна таблица.



Основни типове агенти

Разглеждаме 5 основни типа агенти:

- **Обикновен рефлексен агент** (Simple reflex agents) -- Рефлексните агенти реагират директно на възприятията.
- **Агенти използващи модели** (Model-based reflex agents) -- Агентите използващи модели поддържат вътрешни състояния за да проследяват аспекти на околния свят които не са очевидни за конкретно моментно възприятие.
- **Агенти със зададена цел** (Goal-based agents). Стремят да постигнат зададената конкретна цел.
- **Агенти оптимизиращи очакваната полезност** чрез целеви функции (Utility-based agents) -- използват целеви функции с които могат да оценяват степента на полезност и да се стремят към повишаването на степента на удовлетвореност.
- **Самообучаващи се агенти**

Рефлексни агенти

Обикновен рефлексен агент

(Simple reflex agents)

Рефлексните агенти реагират директно на възприятията.

Рефлексни агенти - Reflex agents

function REFLEX-VACUUM-AGENT(*[location,status]*) **returns** an action

if *status* = *Dirty* **then return** *Suck*

else if *location* = *A* **then return** *Right*

else if *location* = *B* **then return** *Left*



Рефлексни агенти - Reflex agents

Percept sequence	Action
$[A, \textit{Clean}]$	<i>Right</i>
$[A, \textit{Dirty}]$	<i>Suck</i>
$[B, \textit{Clean}]$	<i>Left</i>
$[B, \textit{Dirty}]$	<i>Suck</i>
$[A, \textit{Clean}], [A, \textit{Clean}]$	<i>Right</i>
$[A, \textit{Clean}], [A, \textit{Dirty}]$	<i>Suck</i>
$\vdots$	$\vdots$
$[A, \textit{Clean}], [A, \textit{Clean}], [A, \textit{Clean}]$	<i>Right</i>
$[A, \textit{Clean}], [A, \textit{Clean}], [A, \textit{Dirty}]$	<i>Suck</i>
$\vdots$	$\vdots$

Figure 2.3 Partial tabulation of a simple agent function for the vacuum-cleaner world shown in Figure 2.2.



Рефлексни агенти - Reflex agents

Базиран на система от правила, дефинирани чрез

- **условие - правило за действие** (condition–action rules),
- **ако ... тогава ... правила** (if–then rules),
- **ситуация - правило за действие** (situation–action rules),
или
- **продукционна логика на действие** (productions).

Обикновен рефлексен агент – Simple reflex agents

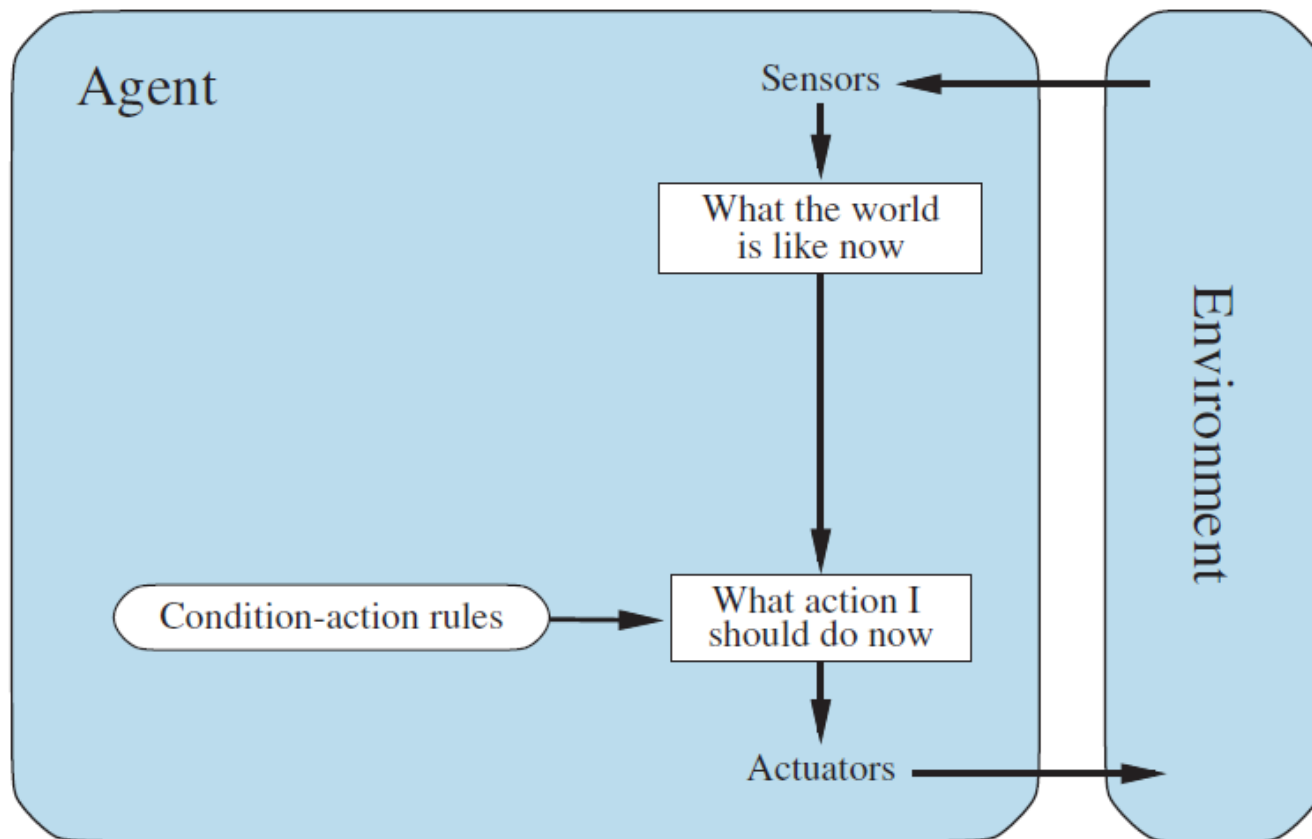


Figure 2.9 Schematic diagram of a simple reflex agent. We use rectangles to denote the current internal state of the agent's decision process, and ovals to represent the background information used in the process.

Обикновен рефлексен агент – Simple reflex agents

function SIMPLE-REFLEX-AGENT(*percept*) **returns** an action
persistent: *rules*, a set of condition–action rules

state $\leftarrow$ INTERPRET-INPUT(*percept*)

rule $\leftarrow$ RULE-MATCH(*state*, *rules*)

action $\leftarrow$ *rule*.ACTION

return *action*

- INTERPRET-INPUT – създава абстрактно описание на текущото състояние въз основа на възприетата информация.
- RULE-MATCH – връща първото правило от набора което съвпада с зададеното състояние.

Обикновен рефлексен агент – Simple reflex agents

Предимства:

- Лесни за реализация
- Изискват малко ресурси

Недостатъци:

- Риск от „зацикляне“, попадане в безкраен цикъл
- Ограничена възможност за работа при промяна на условията/средата
- Ограничена интелигентност

Агенти използващи модели

(Model-based reflex agents)

Агентите използващи модели поддържат вътрешни състояния за да проследяват аспекти на околния свят които не са очевидни за конкретно моментно възприятие.



Агенти използващи модели

Model-based reflex agents

Основна идея: *да се следи останалата част от света, която не може да се наблюдава в момента, т.е.*

Агентът трябва да поддържа представа за „модел на света“, т.е. **вътрешно състояние**, което зависи от историята на възприятията, с цел да се моделират и използват някои аспекти, които не могат да бъдат наблюдаване в момента.

Агенти използващи модели

Model-based reflex agents

За актуализация на вътрешното състояние на агента се изискват два вида „знание“:

- информация „Как се променя света?“ (transition models)
 - информация "Как светът се развива независимо от агента?" и
 - разбиране "Как действията на агента влияят на света?„
- Как възприятието на агента отразява състоянието на света (sensor model).

Заедно преходният модел и сензорният модел (transition & sensor models) позволяват на агента да следи състоянието на света — доколкото е възможно, предвид ограниченията на сензорите на агента.

Рефлексни агенти използващи модели

Model-based reflex agents



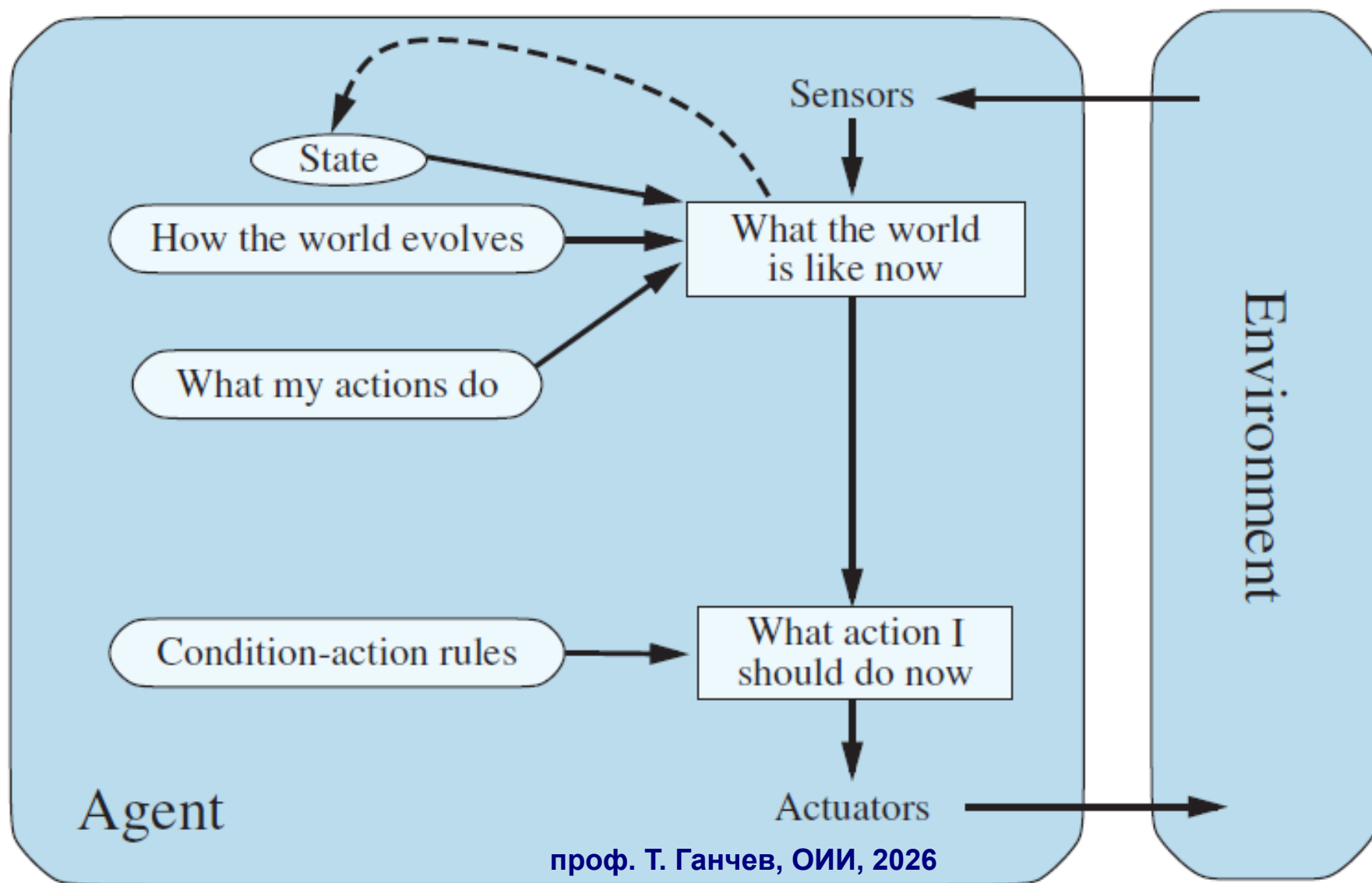
В частично наблюдаем свят обикновено търсим някакво „приблизително“ текущо състояние,

т.е. агента често работи в условия на “best guess” (или даже „best guesses“, когато агента има възможност да вземе под внимание множество възможности).

Необходимо е агента да се справя с несигурност и непълнота дължаща се на частичната наблюдаемост на света.

Рефлексни агенти използващи модели

Model-based reflex agents



Рефлексни агенти използващи модели

Model-based reflex agents



function MODEL-BASED-REFLEX-AGENT(*percept*) **returns** an action

persistent: *state*, the agent's current conception of the world state

model, a description of how the next state depends on current state and action

rules, a set of condition–action rules

action, the most recent action, initially none

state $\leftarrow$ UPDATE-STATE(*state*, *action*, *percept*, *model*)

rule $\leftarrow$ RULE-MATCH(*state*, *rules*)

action $\leftarrow$ *rule*.ACTION

return *action*

Рефлексни агенти използващи модели

Model-based reflex agents



Предимства

- ✓ Възможност за допълване на липсваща информация чрез използване на модел, в конкретен контекст
- ✓ Възможност за частично справяне с несигурност и непълнота на възприятията (в конкретен контекст), дължащ се на частичната наблюдаемост на света

Недостатъци

- Моделът трябва да обхваща всички важни аспекти на света, релевантни към решаваната задача и конкретен контекст.
- Трудна адаптация на правилата и моделите при промяна на задачата или при работа в бързо променяща се среда.
- Ограничена интелигентност (фиксирано поведение, сходно на автомат за конкретен тип задача).
- Ограничени възможности за справяне с несигурност и непълнота на информацията

Тема 13: Агенти
изпълняващи зададен цел.
Агенти оптимизиращи
целева функция.
Самообучаващи се агенти.



Goal-based agents

Агенти изпълняващи зададена цел

Стремят да постигнат зададената конкретна цел.

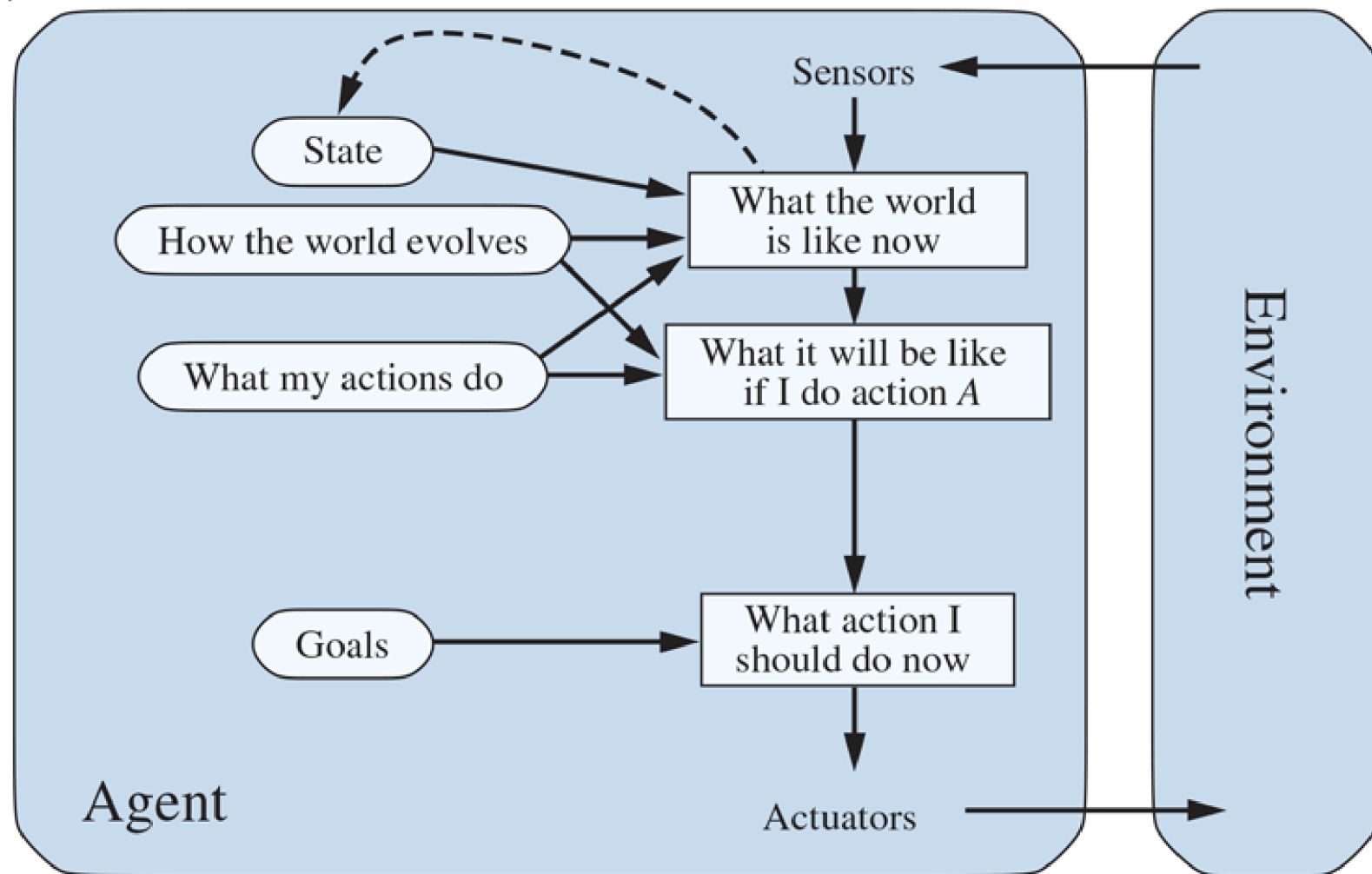
Goal-based agents

Основни концепции при агентите реализиращи зададена цел са:

- Ясна дефиниция на крайната цел – например дестинацията до която трябва да се достигне.
- Търсене на последователността от състояния през които трябва да се премине.
- Планиране (обмисляне на възможностите)
- Вземане на решения предвид възможностите предоставяни в бъдещето.
- Решенията трябва да удовлетворяват стремежа да се постигне конкретна цел, фокусът вече не е върху моментното възприятие.

Model-based, Goal-based agents

48



A model-based, goal-based agent. It keeps track of the world state as well as a set of goals it is trying to achieve, and chooses an action that will (eventually) lead to the achievement of its goals.

Model-based, Goal-based agents



ФОРМУЛИРАНЕ НА ПРОБЛЕМА: Агентът създава описание на състоянията и действията, необходими за постигане на целта — абстрактен модел на съответната част на света.

Един добър пример е да се вземат предвид действията при преместване от едно място на съседно място, и следователно единственият факт за състоянието на света, който ще се промени поради дадено действие, е текущото местоположение.

ТЪРСЕНЕ: Преди да предприеме каквото и да е действие в реалния свят, агентът симулира последователности от действия в своя модел, търсейки докато намери последователност от действия, която достига целта. Такава последователност се нарича решение. Агентът може да се наложи да симулира множество последователности, които не достигат целта, но в крайна сметка (надяваме се) ще намери решение.

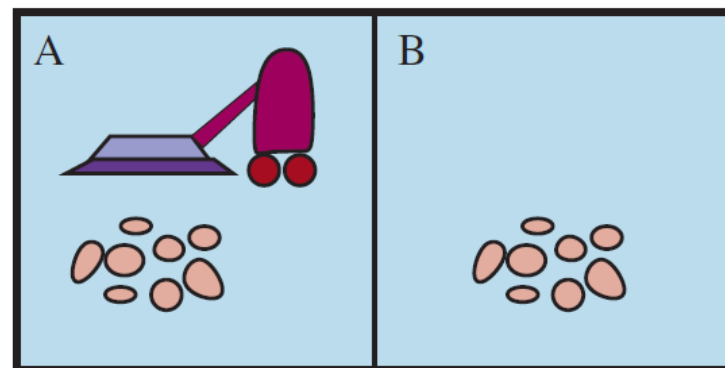
ИЗПЪЛНЕНИЕ: Агентът вече може да изпълнява действията заложи в плана за намиране на решението, едно по едно.

Стандартизирани и реални проблеми

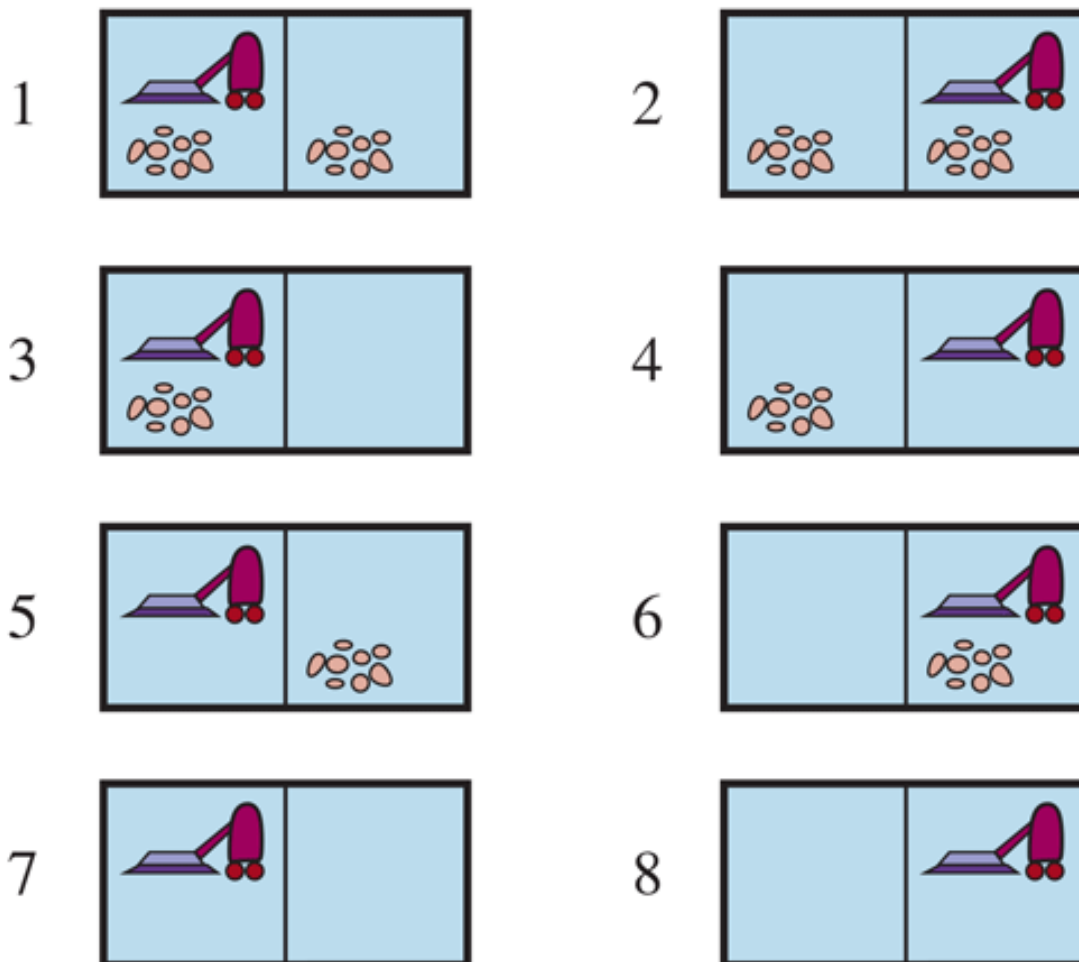


Правим разлика между стандартизирани и реални проблеми.

- **Стандартизираният проблем** има за цел да илюстрира или сравнява различни методи за решаване на проблеми. Така може да се получи кратко и точно описание на задачата и затова е подходящ като ориентир при сравнение на производителността на алгоритмите.
- **Реален проблем** (напр. навигацията на роботи), е този, чиито решения се използват при създаването на реални системи и чиято формулировка е идосинкратична, а не стандартизирана, защото, например, всеки робот има различни сензори, които произвеждат различни данни.



Прахосмукачката



Съществуват осем възможни състояния на разглеждания свят;
състояния 7 и 8 съответстват на постигната цел.



Стандартизиран проблем

При стандартизирания проблем, **светът се разглежда като мрежа**, т.е. двумерен правоъгълен масив от квадратни клетки, в който агентите могат да се движат от клетка на клетка.

Обикновено агентът може да се придвижи към всяка съседна клетка без препятствия — хоризонтално или вертикално, а в някои задачи и диагонално.

Клетките могат да съдържат обекти, които агентът може да вземе, бута или по друг начин да действа; Стена или друго непреодолимо препятствие в клетката пречи на агента да проникне в тази клетка

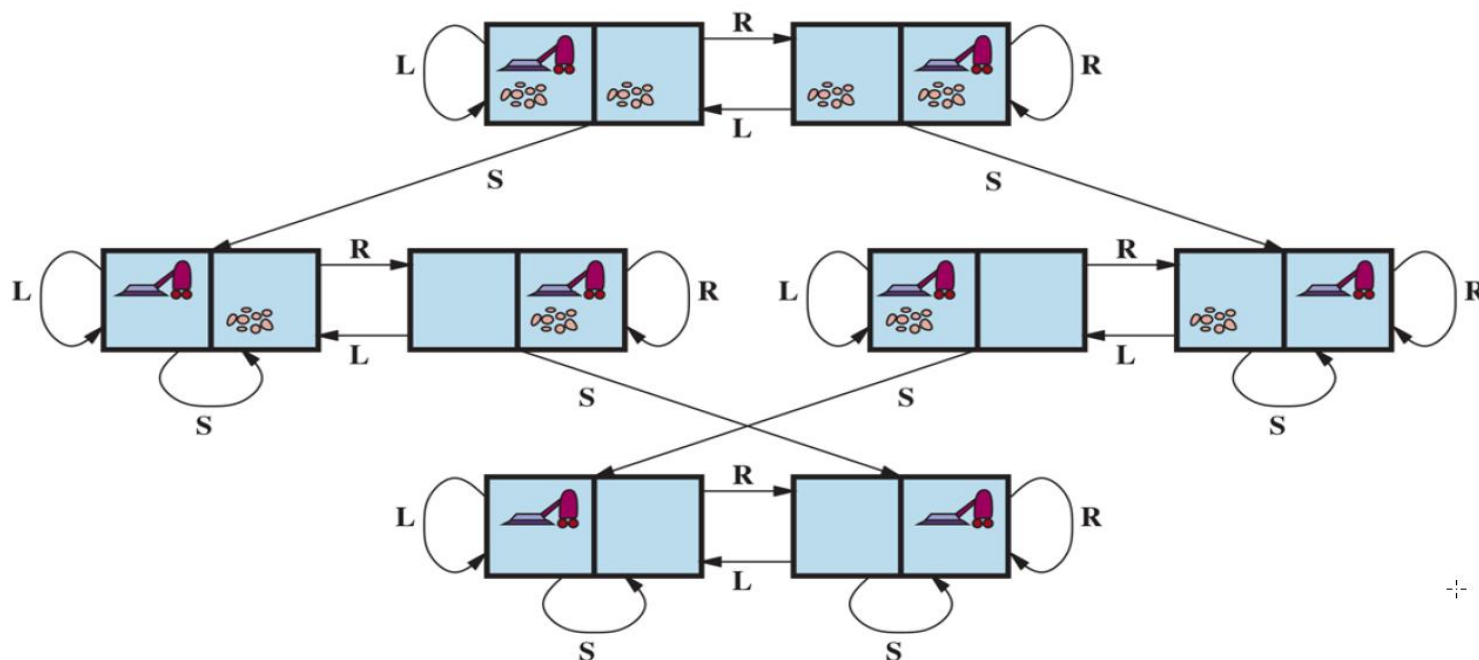
При прахосмукачката, целта на агента е да се почисти цялата мръсотия (състояния 7 и 8), така че **ако средата е напълно наблюдаема, детерминистична и напълно позната**, проблемът е лесен за решаване с който и да е от информираните алгоритми за търсене, а решението се свежда до последователност от действия.

Стандартизиран проблем

СЪСТОЯНИЯ: Състоянието на света казва кои обекти са в кои клетки. За света на прахосмукачката обектите са агентът и всяка мръсотия.

В опростената дву-позиционна версия на света, агентът може да бъде в една от двете клетки, като всяка позиция може да съдържа или не отпадъци, така че имаме общо 8 възможни състояния ($2 \times 2 \times 2$).

В общия случай среда с 'n' клетки има 'n.2ⁿ' състояния.



The state-space graph for the two-cell vacuum world. There are 8 states and three actions for each state: L = *Left*, R = *Right*, S = *Suck*.



Стандартизиран проблем

- **НАЧАЛНО СЪСТОЯНИЕ:** Всяко състояние може да бъде определено като начално състояние.
- **ДЕЙСТВИЯ:** В дву-позиционен свят с три действия: Почистване, движение наляво и надясно.

В двумерен многоклетъчен свят ни трябва повече команди за движение. Можем да добавим Нагоре и Надолу, давайки ни четири абсолютни действия на движение, или можем да преминем към егоцентрични действия, дефинирани спрямо гледната точка на агента — например, *напред, назад, завий надясно и завий наляво*.



Стандартизиран проблем

- **МОДЕЛ НА ПРЕХОД:** *Чистенето премахва всяка мръсотия от позицията на агента; Напред премества агента с една клетка напред в посоката, в която е обърнат, освен ако не удари стена, в който случай действието няма ефект. Обратно движи агента в обратна посока, докато завий надясно и завий наляво променят посоката, в която е обърнат агента с 90°*
- **ЦЕЛЕВИ СЪСТОЯНИЯ:** Състоянията, в които всички клетки са чисти.
- **РАЗХОД ПРИ ДЕЙСТВИЕ:** Всяко действие струва 1.

Реални проблеми

Автономните таксиметрови или автомобилни навигационни системи ще предоставят указания за шофиране, решаващи проблема „Турне“ (набор от локации, които трябва да бъдат посетени, а не една единствена целева дестинация).

Екстремален случай: Проблемът с пътуващия търговец (TSP) е проблем, при който всеки град на картата трябва да бъде посетен само веднъж. Целта е да се обходи маршрута с минимални разноси (или в оптимизационната версия, да се намери пълното турне с възможно най-ниска цена).



Оценяване на производителността при решаване на проблеми

Можем да оценим производителността на алгоритъма по четири критерия:

- **ПЪЛНОТА:** Гарантирано ли е, че алгоритъмът ще намери решение, когато има такова, и ще докладва правилно неуспеха, когато няма.
- **ОПТИМАЛНОСТ НА РАЗХОДИТЕ:** Намира ли решение с най-ниски пътни разходи от всички решения? (Някои автори използват термина "допустимост" за свойството да намерят най-евтиното решение, а други използват само "оптималност", но това може да се обърка с други видове оптималност.
- **ВРЕМЕВА СЛОЖНОСТ:** Колко време отнема да се намери решение? Това може да се измери в секунди или по-абстрактно чрез броя на разглежданите състояния и действия.
- **ПРОСТРАНСТВЕНА СЛОЖНОСТ:** Колко памет е необходима за извършване на търсенето?

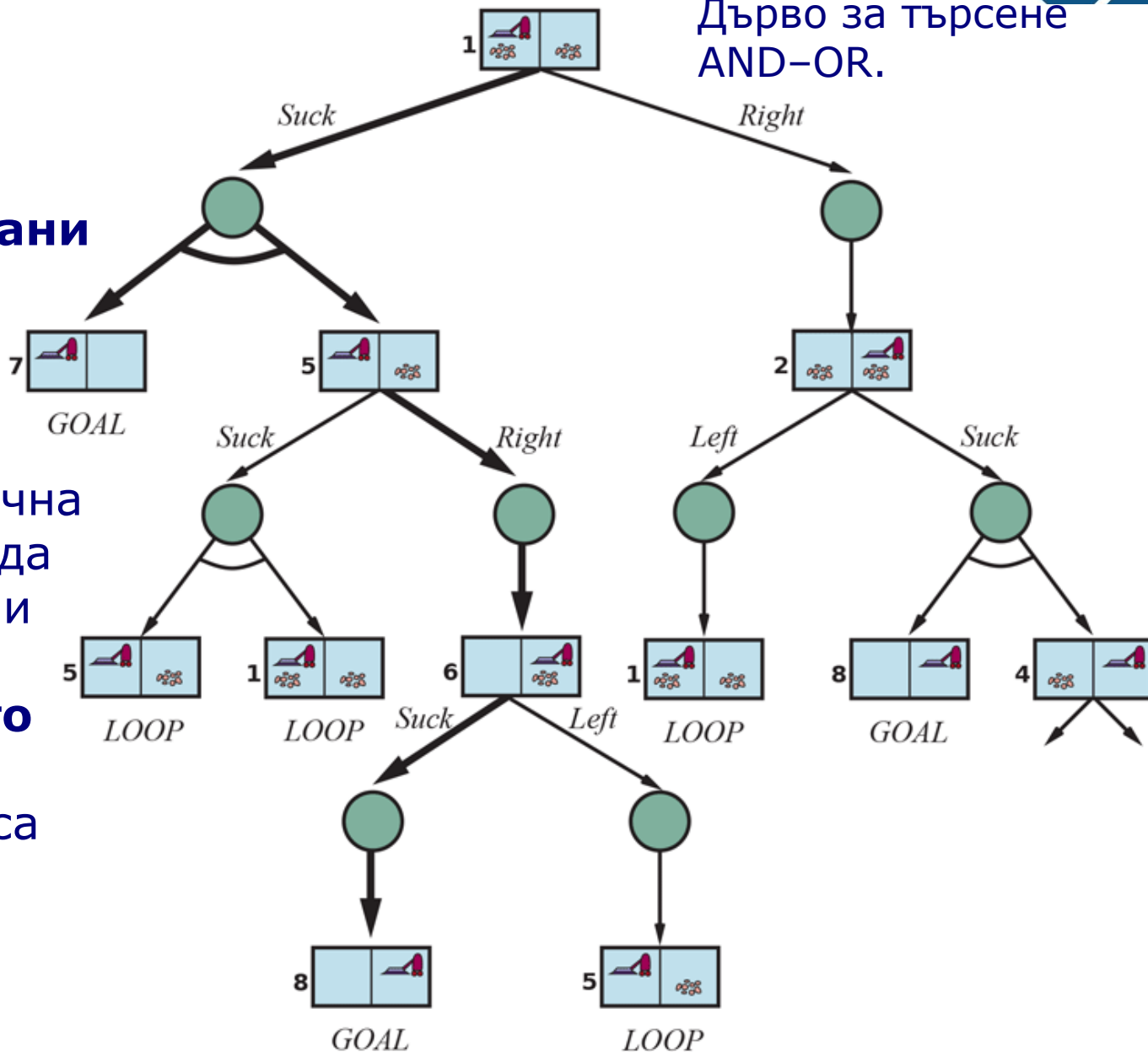


Непредсказуемият свят

Как намираме
контингентни
решения на
недетерминирани
проблеми?

В една
недетерминистична
среда, се въвежда
разклоняване и
чрез **избора на**
средата на като
ИЗХОД за всяко
действие. Това са
възлите AND.

Дърво за търсене
AND-OR.



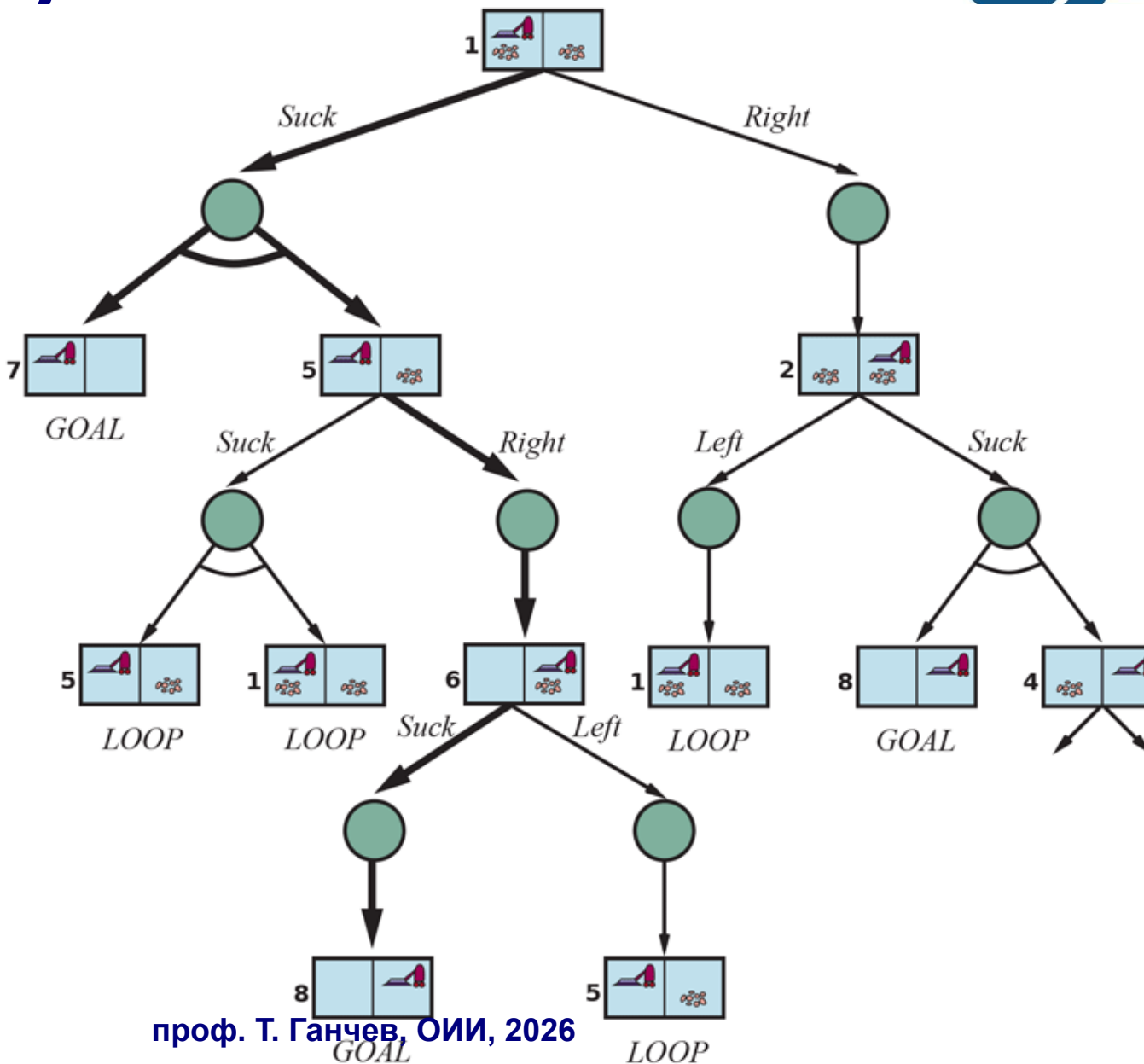
В AND възлите
(зелени кръгове)
всеки резултат
трябва да бъде
обработен, както е
посочено
чрез дъгата,
свързваща
изходящите клони.
Тук намерено
решение е
показано с
удебелени редове.



Непредсказуемият свят

Решение на задачата за търсене AND-OR е под-дърво на пълното дърво на търсенето, което:

- (1) Има целеви възел (GOAL) на всяко листо,
- (2) Задава по едно действие във всеки от своите OR възли, и
- (3) Включва всеки резултат при преходи в своите AND възли.





Непредсказуемият свят

An AND-OR search tree algorithm

function AND-OR-SEARCH(*problem*) **returns** a conditional plan, or *failure*
return OR-SEARCH(*problem*, *problem*.INITIAL, [])

function OR-SEARCH(*problem*, *state*, *path*) **returns** a conditional plan, or *failure*
if *problem*.IS-GOAL(*state*) **then return** the empty plan
if IS-CYCLE(*path*) **then return failure**
for each *action* **in** *problem*.ACTIONS(*state*) **do**
 plan $\leftarrow$ AND-SEARCH(*problem*, RESULTS(*state*, *action*), [*state*] + *path*)
 if *plan* $\neq$ *failure* **then return** [*action*] + *plan*
return failure

function AND-SEARCH(*problem*, *states*, *path*) **returns** a conditional plan, or *failure*
for each s_i **in** *states* **do**
 plan_i $\leftarrow$ OR-SEARCH(*problem*, s_i , *path*)
 if *plan_i* = *failure* **then return failure**
return [if s_1 **then** *plan₁* **else if** s_2 **then** *plan₂* **else** ... **if** s_{n-1} **then** *plan_{n-1}* **else** *plan_n*]

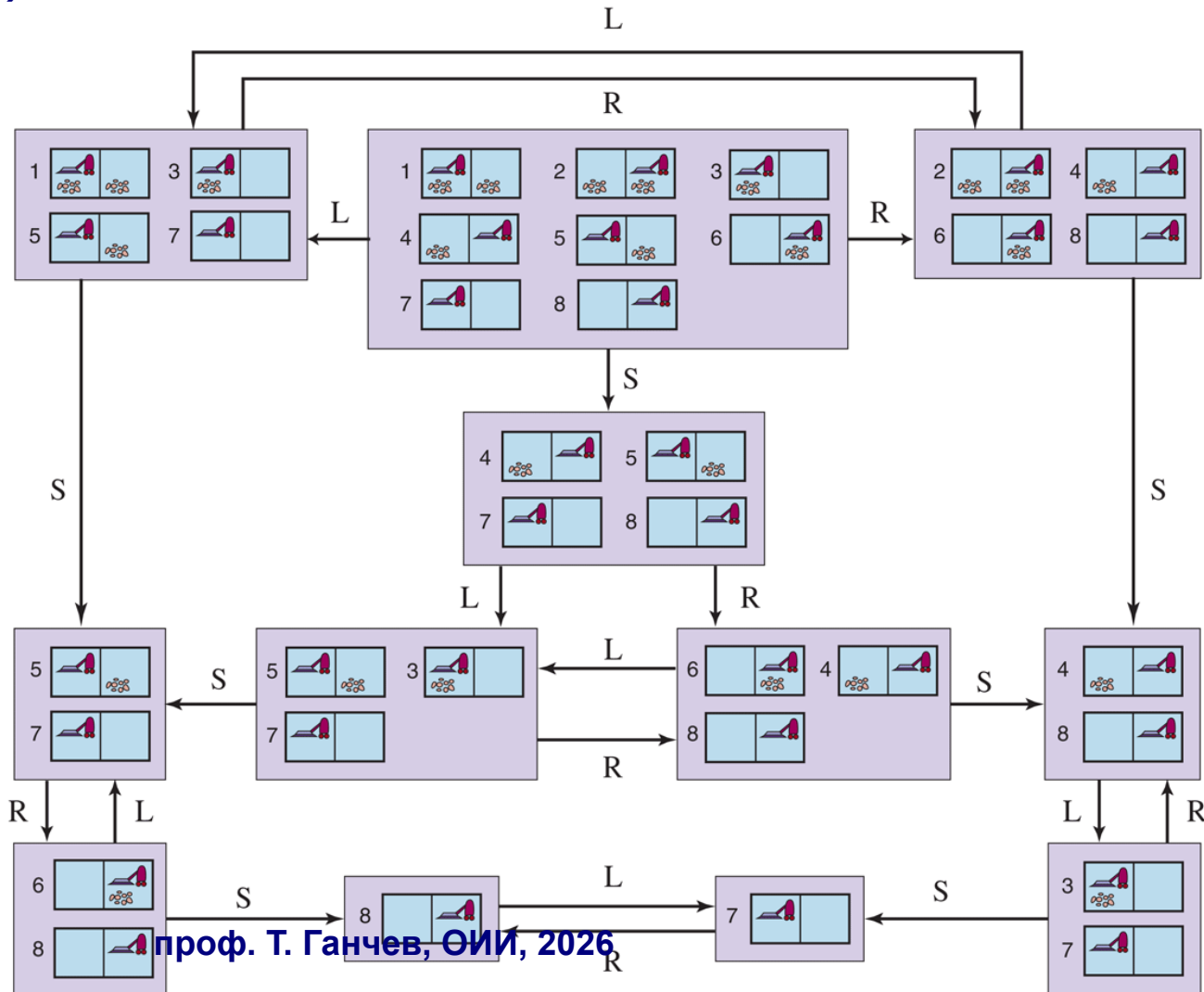


Частично наблюдаеми среди

Търсене без наблюдение – когато възприятията на агента не предоставят *никаква информация* (без сензори, известен още като конформънт проблем).

Достижимото пространство на състояния на вярвания за детерминистични, безсензорен свят на прахосмукачката.

Има само 12 достижими състояния на вярване от 256 възможни.



Частично наблюдаеми среди

Прахосмукачка в частично наблюдаема среда:

- Първо, решението ще бъде условен план, а не последователност; За да изпълни if-then-else израз, агентът трябва да тества условието и да изпълни съответния клон на условието.
- Второ, агентът трябва да поддържа състоянието си на убеждение, докато извършва действия и получава възприятия.

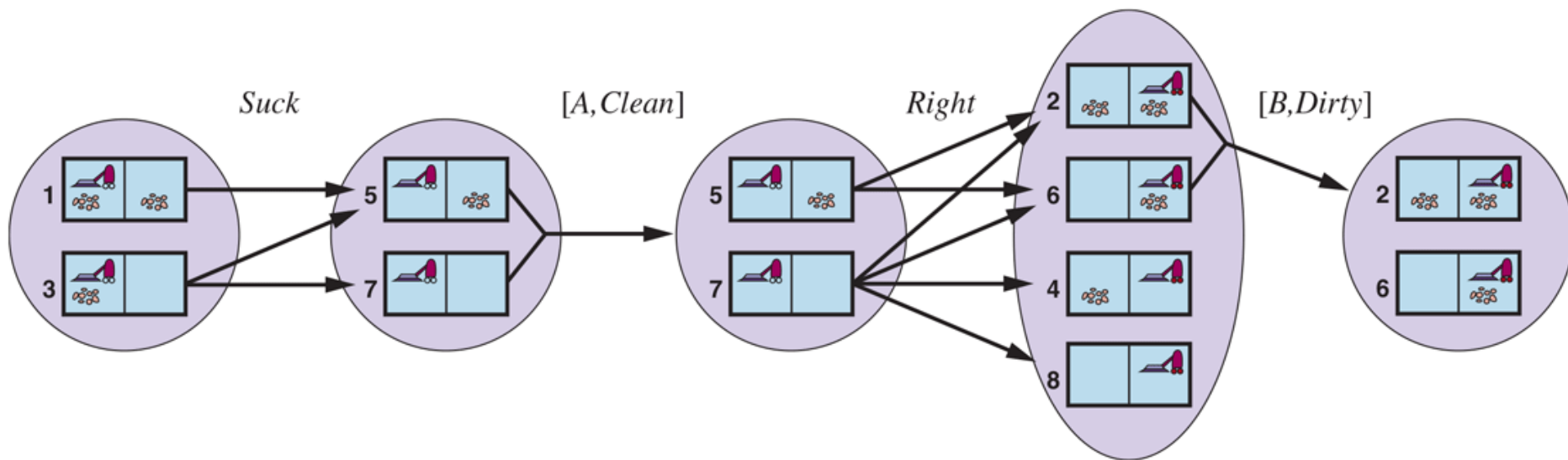
Тук има опростяване, защото възприятието се дава от средата, а не се изчислява от агента.

Дадено начално състояние на вярване b , действие a и възприятие o , новото състояние на вярване e :

$$b' = \text{UPDATE}(\text{PREDICT}(b, a), o) .$$

Частично наблюдаеми среди

Два цикъла на предсказване и актуализация на поддържане на състоянието на вяра в опростения свят на прахосмукачката с локални възприятия.



В частично наблюдаеми среди (повечето реални среди) поддържането на състоянието на вяра е основна функция на всяка интелигентна система.

Тази функционалност е позната под различни имена, включително **мониторинг, филтриране и оценка на състоянието**.

Model-based, Goal-based agents

Предимства

- ✓ Подобрена гъвкавост при вземане на решенията, поради наличието на знание за крайната цел – внася се някакъв елемент на осъзнатост и далновидност.
- ✓ Възможност за модификация на поведението при коригиране на крайната цел.

Недостатъци

- Ограничена интелигентност при зле структурирана задача, при която решението не може да се сведе до тривиална последователност от конкретни действия, които агента може да извърши.
- Моделът трябва да обхваща всички важни аспекти на света, релевантни към решаваната задача и конкретен контекст
- Трудна адаптация на моделите при бързо променяща се среда

Агенти оптимизиращи очакваната полезност
Агенти използващи целеви функции
(Utility-based agents) –

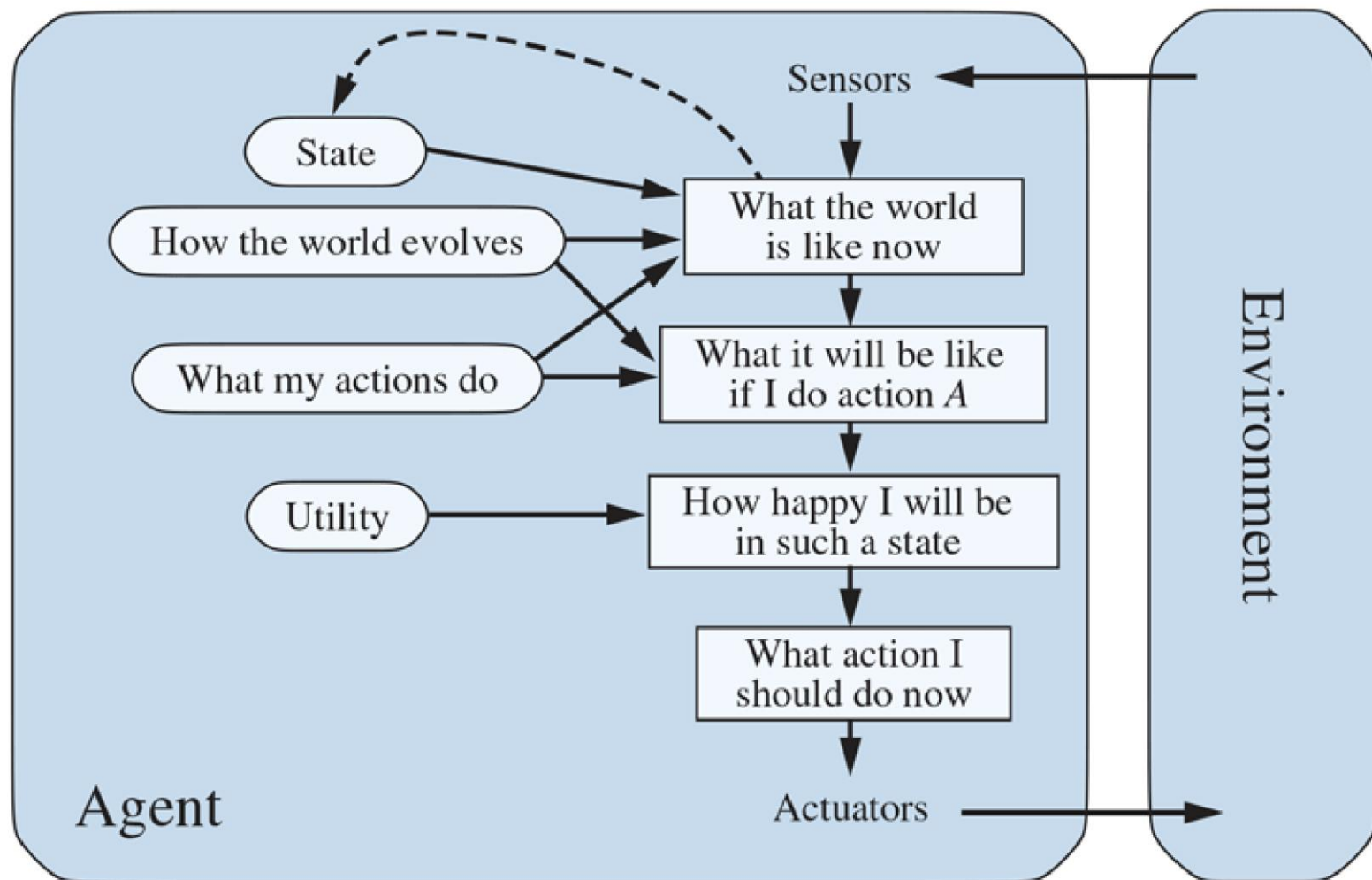
могат да оценяват степента на полезност и да се стремят към повишаването на степента на удовлетвореност.

Utility-based agents

Задаването на цел не води (автоматично) до смислени (висококачествени) поведения на агента:

- Достигането/недостигането на целта е бинарен проблем
- Възможно е да има множество начини за достигане на целта, кой е най-добър (множество критерии, често взаимоизключващи се).
- Намерените решения трябва да удовлетворяват допълнителни ограничения, които да се вземат предвид при приемането на компромиси.
- Оптимизира се очакваната полезност, независимо дали крайната цел може да бъде достигната или не.
- Рационалност, съгласно дефинираната целева функция и мярката за успешност.
- Желано поведение за всеки рационален агент се свежда до възможностите определени от локалните ограничения.

Utility-based agents



A model-based, utility-based agent. It uses a model of the world, along with a utility function that measures its preferences among states of the world. Then it chooses the action that leads to the best expected utility, where expected utility is computed by averaging over all possible outcome states, weighted by the probability of the outcome.

Utility-based agents

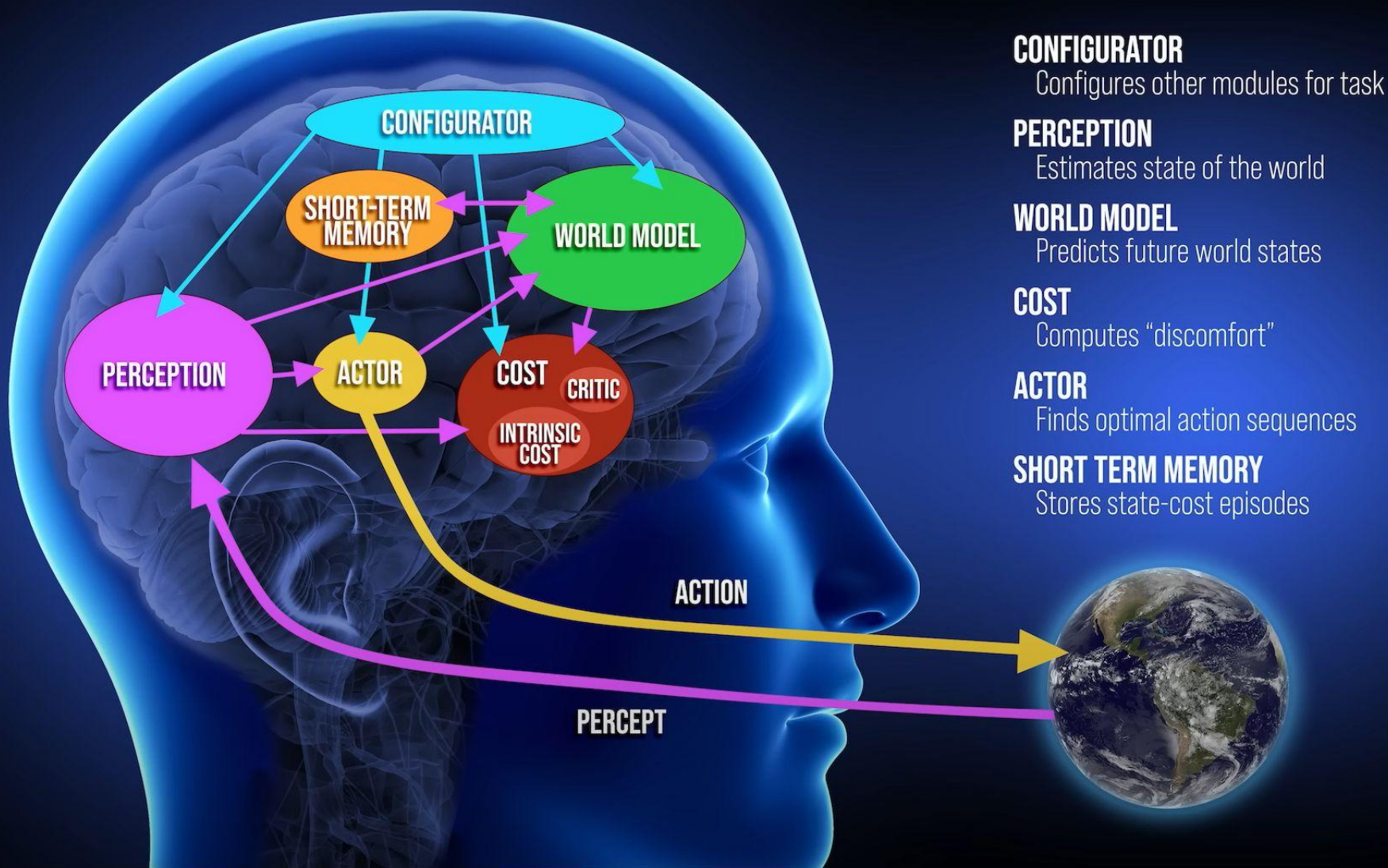
Предимства

- ✓ Подобрена гъвкавост при вземане на решенията, поради наличието на собствена мярка за полезност – което води до някаква степен на осъзнатост и далновидност.
- ✓ Подобрени възможности за „адаптиране/научаване“ на нови неща. Възможност за модификация на поведението при коригиране на целевата функция, която дефинира мярката за полезност.
- ✓ Подобрена устойчивост в условия на несигурност.
- ✓ Възможност за постигане на няколко цели едновременно.
- ✓ „Очаквана полезност“

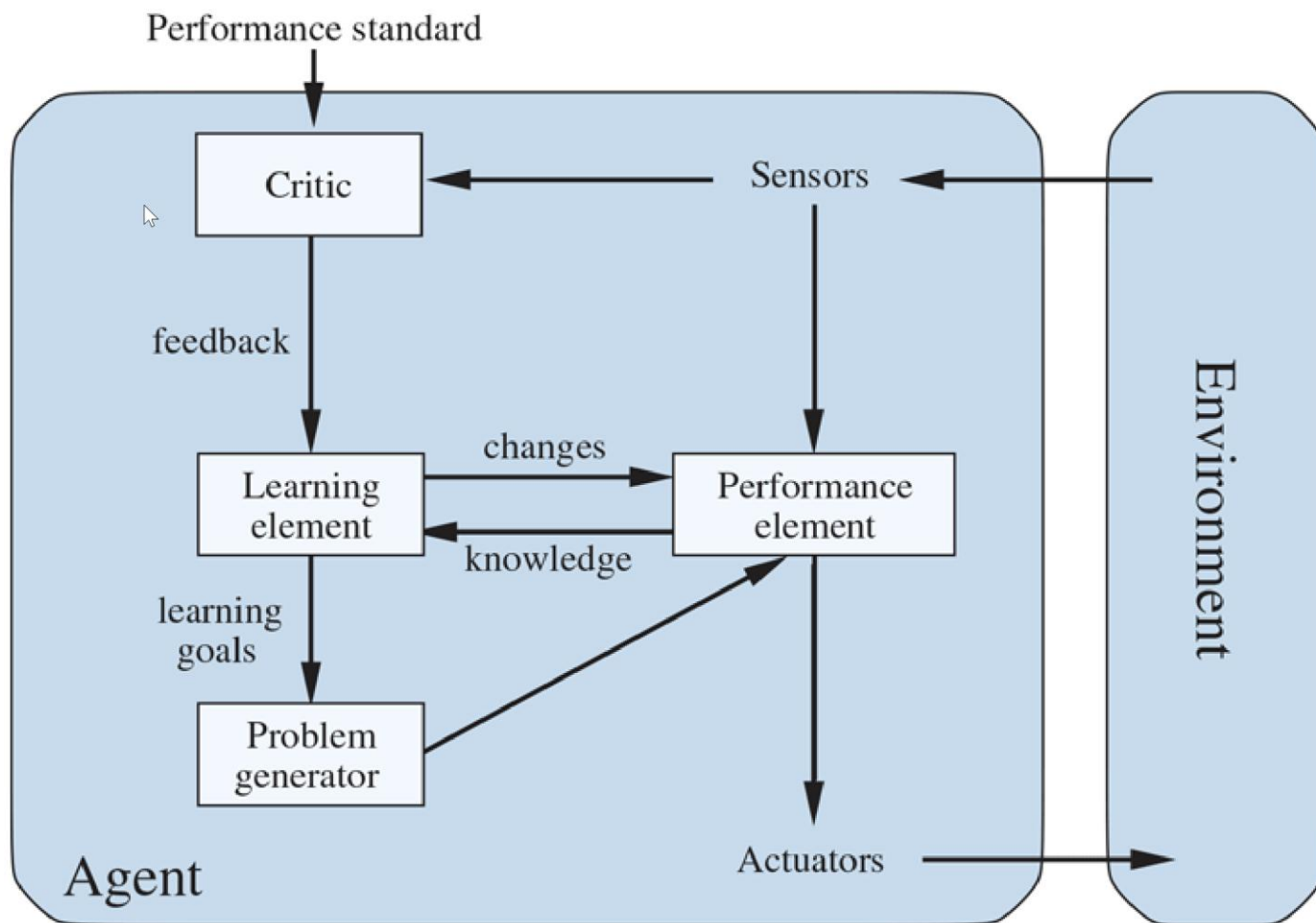
Недостатъци

- Ограничена интелигентност при зле структурирана задача.
- Чувствителност към „малки промени“ в целевата функция
- Моделът трябва да обхваща всички важни аспекти на света, релевантни към решаваната задача и конкретен контекст
- Трудна адаптация на моделите при бързо променяща се среда

Самообучаващи се агенти (Learning agents)

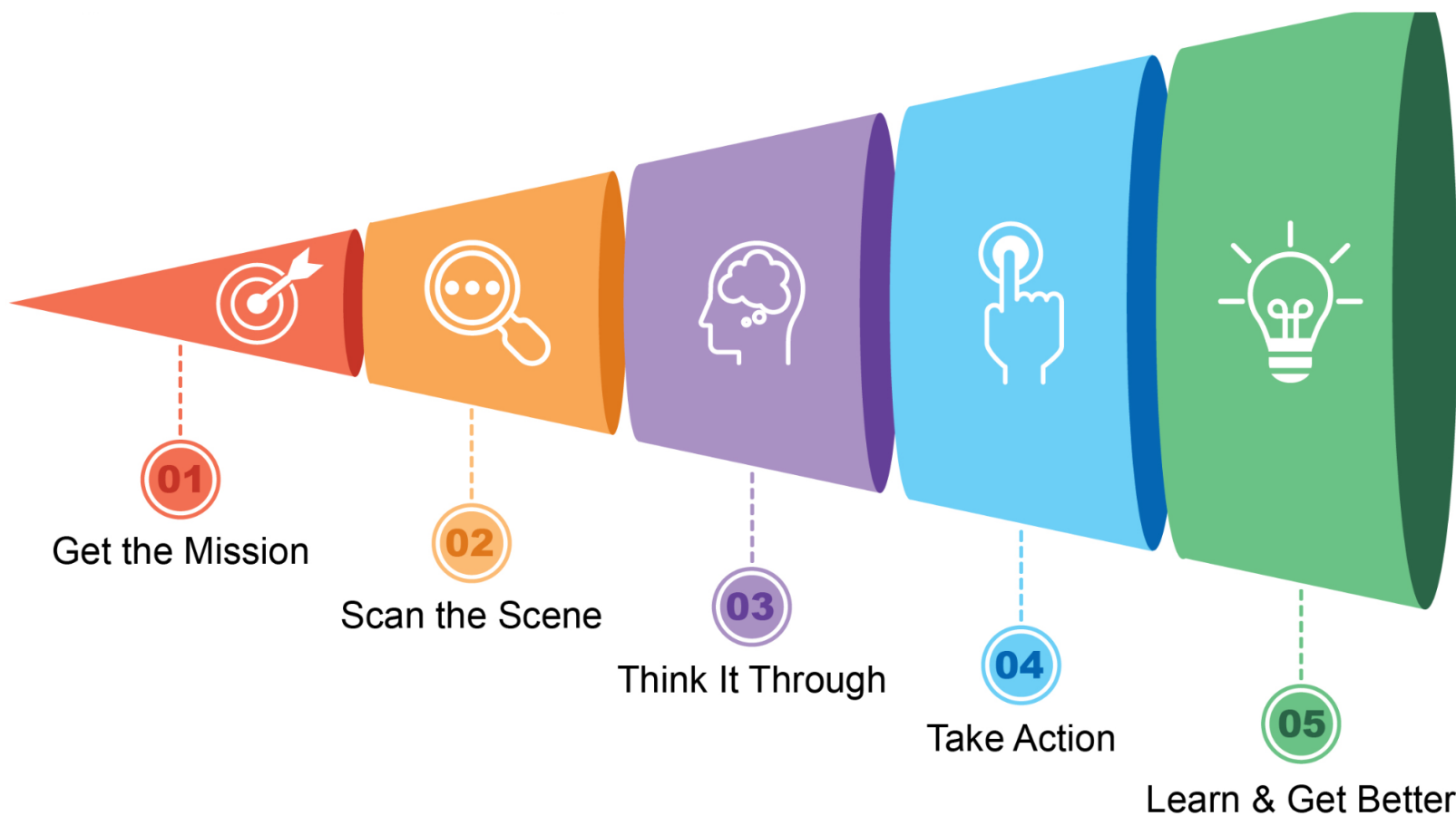


Самообучаващи се агенти (Learning agents)

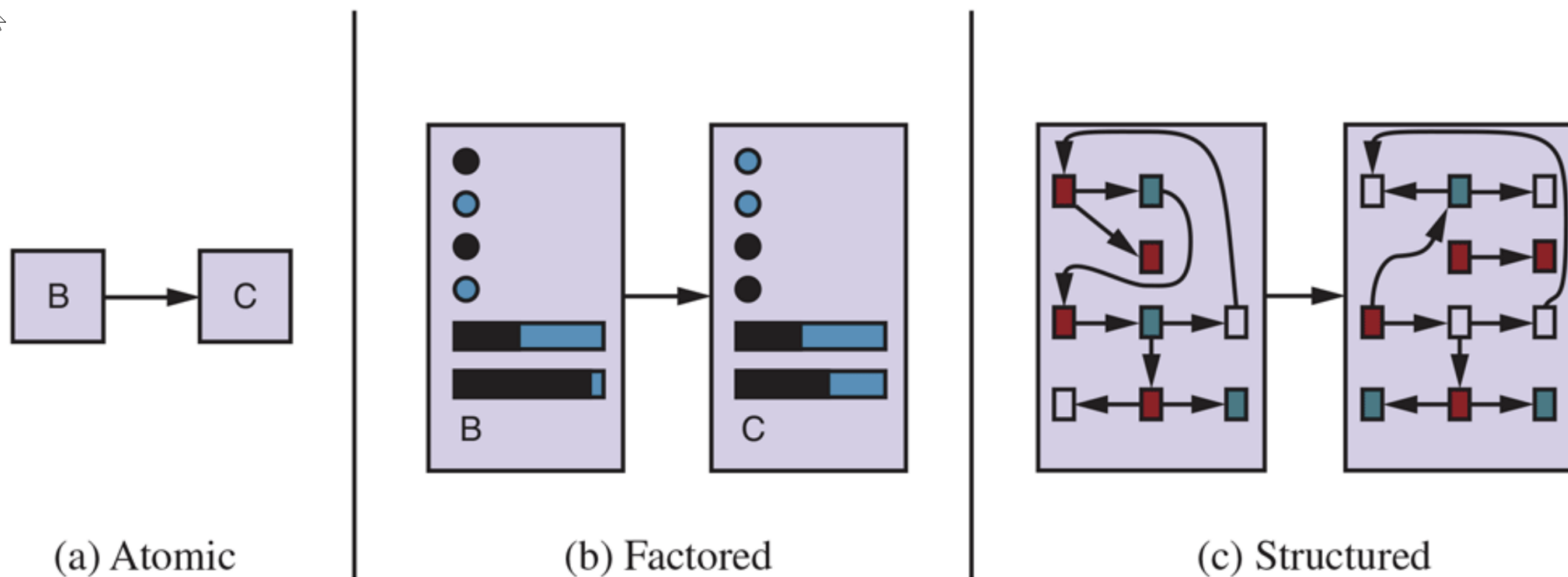


A general learning agent. The “performance element” box represents what we have previously considered to be the whole agent program. Now, the “learning element” box gets to modify that program to improve its performance.

Функционалности, реализирани в самообучаващите се агенти



Представяне на вътрешните състояния



Three ways to represent states and the transitions between them. (a) Atomic representation: a state (such as B or C) is a black box with no internal structure; (b) Factored representation: a state consists of a vector of attribute values; values can be Boolean, real-valued, or one of a fixed set of symbols. (c) Structured representation: a state includes objects, each of which may have attributes of its own as well as relationships to other objects.

Въпроси?

